ICLR 2025 · APPLE

# Cut Your Losses in Large-Vocabulary Language Models

Erik Wijmans, Brody Huval, Alexander Hertzberg, Vladlen Koltun, Philipp Krähenbühl



**Статья на arXiv**

2411.09009

# LLM и память

УЖЕ РЕШИЛИ

**Веса и оптимизаторы** → ☑ **ZeRO / FSDP**
шардирование состояния по GPU

**Квадратичный контекст** → ☑ **FlashAttention**
внимание блоками в SRAM

**Активации** → ☑ **Activation checkpointing**
пересчёт вместо хранения

НАДО БЫЛО РЕШИТЬ

**Cross-Entropy Loss**



Логиты $N \times |V|$ целиком лежат в памяти — и на forward, и на backward

# Проблема классической Cross-Entropy

$N$ – количество токенов в батче

$D$ – размерность эмбеддинга

$|V|$ – размер словаря токенизатора

$\mathbf{E} \in \mathbb{R}^{D \times N}$ — матрица эмбеддингов

$\mathbf{C} \in \mathbb{R}^{D \times |V|}$ — веса классификатора

$$A = \mathbf{C}^\top \mathbf{E} \in \mathbb{R}^{|V| \times N}$$

ПОЛНОСТЬЮ МАТЕРИАЛИЗУЕТСЯ В ПАМЯТИ

$$P_{j,n} = \mathrm{softmax}(A)_{j,n} = \frac{\exp(A_{j,n})}{\sum_{k=1}^{|V|} \exp(A_{k,n})}$$

вероятность, что следующий токен для n-го – это j-й элемент словаря

$$\ell_n = \log P_{x_n,n} = \log \frac{\exp(A_{x_n,n})}{\sum_{k=1}^{|V|} \exp(A_{k,n})}$$

log-likelihood правильного токена $x_n$

**90%**

логиты loss-слоя

**10%**

веса, оптимизатор, активации



# Пример

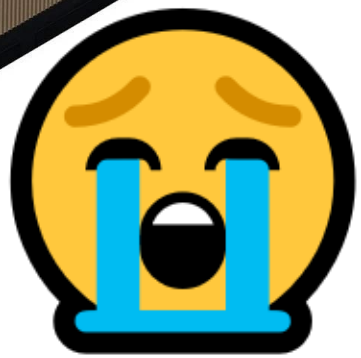
**GEMMA 2 (2B)** одна последовательность длиной 80 000 токенов, логиты в fp32

$$\underbrace{256\,128}_{|V|} \times \underbrace{80\,000}_{N} \times \underbrace{4\ \mathrm{B}}_{\mathrm{fp32}} \approx 81.96 \cdot 10^9\ \mathrm{B} \approx \mathbf{82\ GB}$$

NVIDIA H100

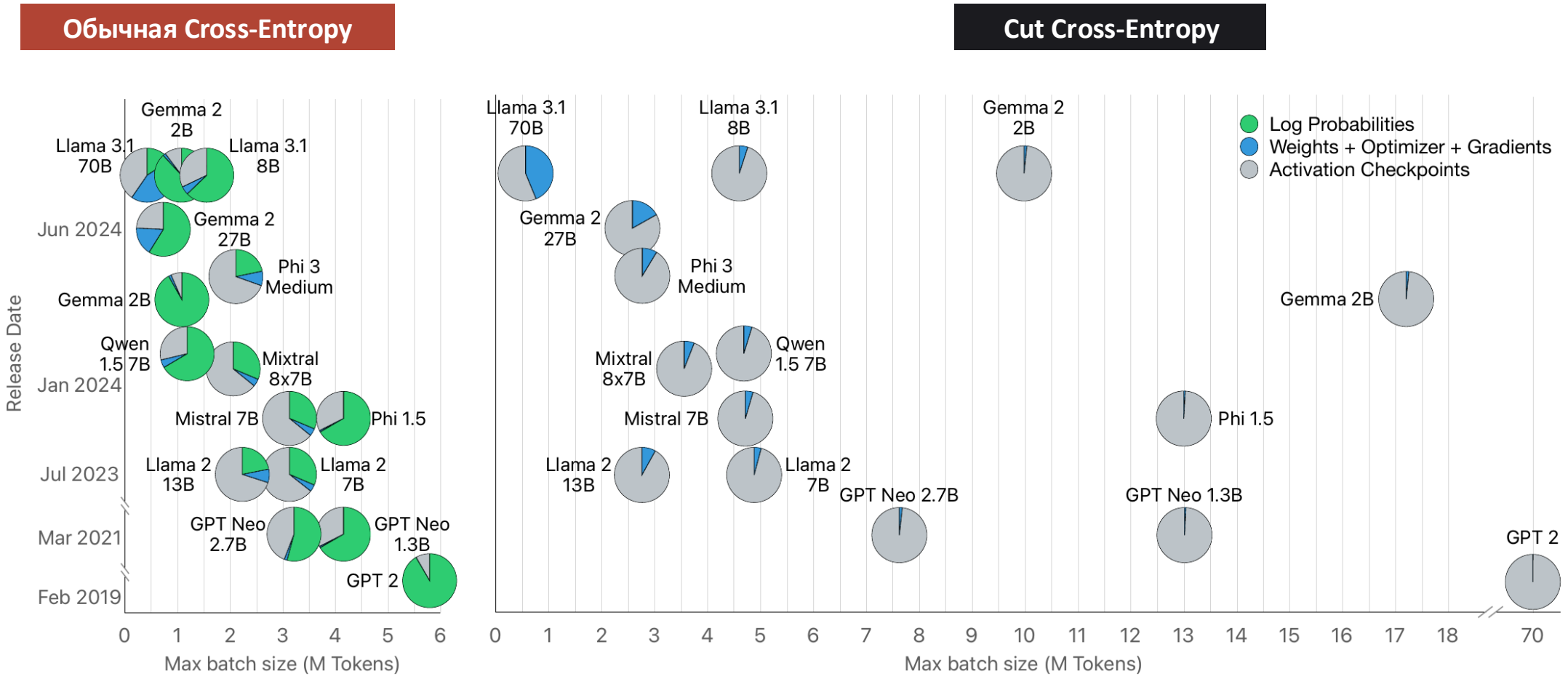
**80 GB**



**Уничтожена одним слоем**

# Наглядный результат




**Зелёное** – память под логиты. По оси X — максимальный batch на 16 GPU. С CCE получается в несколько раз увеличить размер батча.

# Идея Cut Cross-Entropy (CCE)

**Никогда не записывать полную матрицу логитов A в глобальную память (DRAM)**

$$\ell_n = A_{x_n,n} - \log \sum_{k=1}^{|V|} \exp(A_{k,n})$$

$$\ell = \underbrace{(\mathbf{C}^\top \mathbf{E})_x}_{\text{indexed matmul}} - \underbrace{\mathrm{LSE}(\mathbf{C}^\top \mathbf{E})}_{\text{linear log-sum-exp}}$$

**Первое слагаемое $(C^T E)_x$**

Логит только правильного токена: скалярное произведение эмбеддинга $E_n$ и строки весов C для $x_n$. На выходе — вектор длины N, а не матрица $N \times |V|$.

**Второе слагаемое $\mathrm{LSE}(C^T E)$**

Знаменатель softmax: $\log \Sigma \square \exp(C \square^T E_n)$. CCE считает его блоками в быстрой SRAM и никогда не пишет логиты в HBM.

# Who is SRAM?

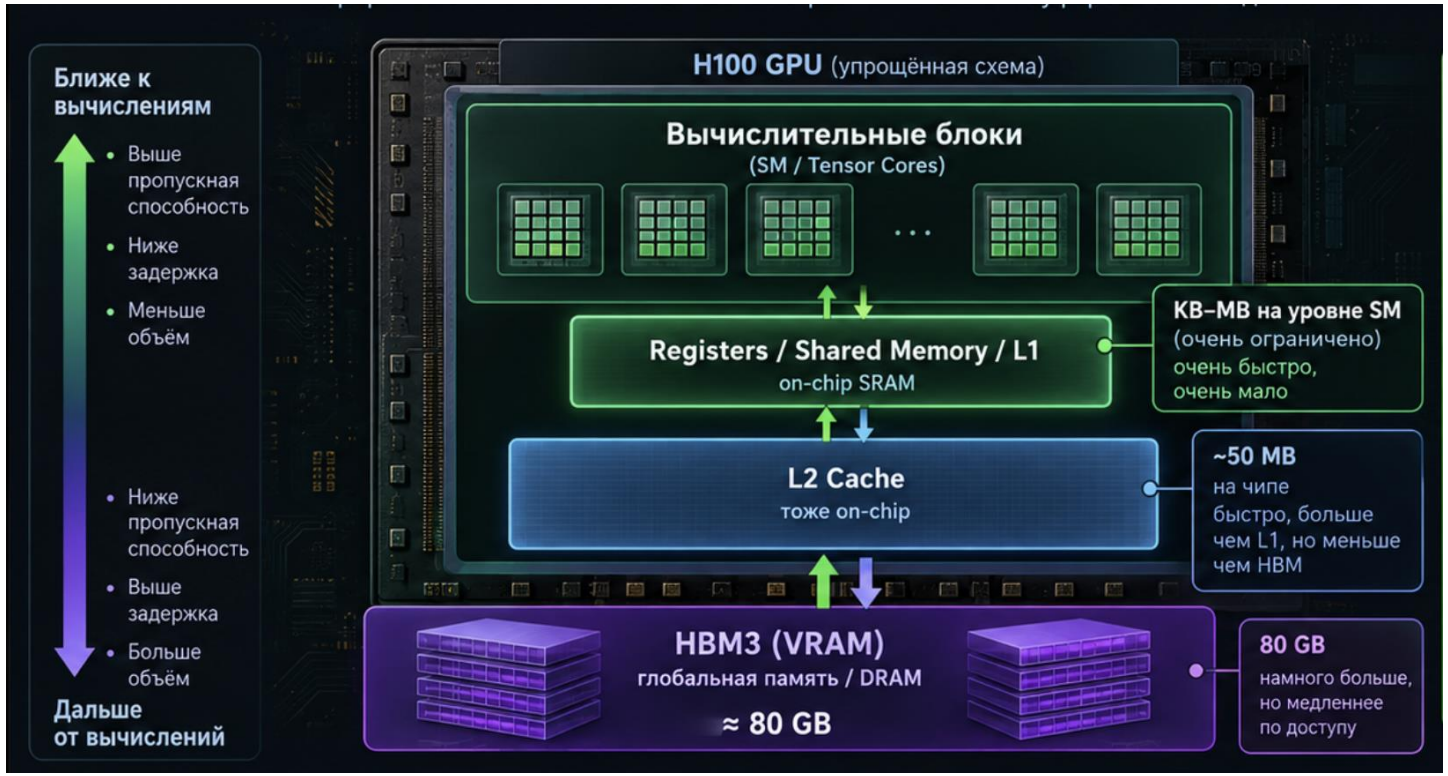
## SRAM

маленькая и очень быстрая память прямо на чипе — KB–MB на каждый SM

## HBM / DRAM

большая, но медленная глобальная память GPU — те самые 80 GB

**CCE, как и FlashAttention, держит блок логитов только в SRAM: посчитали, свернули, выбросили.**



# Алгоритм 1 – расчёт $(C^TE)_x$, forward

**Algorithm 1** Memory-efficient indexed matrix multiplication

**Inputs:** $\mathbf{E} \in \mathbb{R}^{D \times N}$, $\mathbf{C} \in \mathbb{R}^{D \times |V|}$, $\mathbf{x} \in \mathbb{R}^N$.
Block sizes $N_B$ and $D_B$.

**Outputs:** $\mathbf{o} = (\mathbf{C}^\top \mathbf{E})_\mathbf{x} \in \mathbb{R}^N$

```
for blocks E_n, x_n do                ▷ Divide E and x into blocks of size D × N_B and N_B, respectively
    o_n = 0_{N_B}                     ▷ Zero vector of size N_B in on-chip SRAM
    for blocks E_{n,d} do             ▷ Divide E_n into blocks of size D_B × N_B
        c = C_{x_n,d}                 ▷ Indexed load into on-chip SRAM
        o_n += E_{n,d} · c            ▷ Column-wide dot product
    end for
    write o_n                         ▷ From on-chip SRAM to main GPU memory
end for
```

$$(\mathbf{C}^\top \mathbf{E})_{x_n} = \mathbf{C}_{x_n}^\top \mathbf{E}_n$$

логит правильного токена = одно скалярное произведение

**Цель**

Быстро посчитать числитель softmax только для правильных токенов.

**Механика**

Triton-ядро объединяет индексацию и скалярное произведение: читает индекс $x_n$, загружает из HBM в SRAM только строку C для $x_n$ и вектор $E_n$, перемножает и сразу пишет результат.

**Память**

**O(N · |V|) → O(N)**

# Алгоритм 2 – расчёт LSE($C^TE$), forward

**Algorithm 2** Memory-efficient linear-log-sum-exp, forward pass

**Inputs:** $\mathbf{E} \in \mathbb{R}^{D \times N}$ and $\mathbf{C} \in \mathbb{R}^{D \times |V|}$.
Block sizes $N_B$, $V_B$, and $D_B$.
**Outputs:** $\mathrm{LSE} = \log \sum_j \exp(C_j^\top \mathbf{E}) \in \mathbb{R}^N$

$\mathrm{LSE} = -\infty_N$ ▷ $-\infty$ vector of size $N$ in main GPU memory
**for** all pairs of blocks $\mathbf{E}_n$, $\mathbf{C}_v$ **do** ▷ Divide $\mathbf{E}$ and $\mathbf{C}$ into blocks of size $D \times N_B$ and $D \times V_B$
  $\mathbf{A}_{nv} = \mathbf{0}_{V_B \times N_B}$ ▷ Zero matrix of size $V_B \times N_B$ in on-chip SRAM
  **for** blocks $\mathbf{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbf{E}_n$ and $\mathbf{C}_v$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\mathbf{A}_{nv} \mathrel{+}= \mathbf{C}_{v,d}^\top \cdot \mathbf{E}_{n,d}$ ▷ Blockwise matrix multiplication
  **end for**
  $\mathrm{LSE}_{nv} = \log \sum \exp(\mathbf{A}_{nv}^\top)$ ▷ Numerically stable implementation with max
  $\mathrm{LSE}_n = \log(\exp(\mathrm{LSE}_n) + \exp(\mathrm{LSE}_{nv}))$ ▷ Locking thread-safe log-add-exp
**end for**

$$\mathrm{LSE}_n \leftarrow \log\left(e^{\mathrm{LSE}_n} + e^{\mathrm{LSE}_{nv}}\right)$$

частичные LSE сливаются атомарно (spin-lock)

**Цель**

Посчитать знаменатель softmax, не сохраняя матрицу логитов.

**Механика**

1. E и C режутся на небольшие блоки (тайлы)
2. CUDA-блоки параллельно считают $A_{nv} = C_v^T E_n$ в SRAM
3. там же — локальный $LSE_{nv}$ с вычитанием максимума
4. $LSE_{nv}$ атомарно сливаются в общий вектор $LSE_n$ в HBM

# Backward: понимание

1 $\ell = (\mathbf{C}^\top \mathbf{E})_x - \mathrm{LSE}(\mathbf{C}^\top \mathbf{E})$ $(C^TE)_x$ – дешёвый indexed matmul backward. **LSE** – вот тут уже тяжело.

2 $\nabla \mathbf{E} = \lambda^\top \frac{\partial\, \mathrm{LSE}}{\partial \mathbf{E}}, \qquad \nabla \mathbf{C} = \lambda^\top \frac{\partial\, \mathrm{LSE}}{\partial \mathbf{C}}, \qquad \lambda = \nabla \mathrm{LSE}$

3 $\mathrm{LSE}_n = \log \sum_{j=1}^{|V|} \exp\left((\mathbf{C}^\top \mathbf{E})_{j,n}\right)$ для конкретного токена n; $(C^TE)_{j,n} = C_j^T E_n$ — логит j-го слова

4 $\mathbf{S}_{k,n} = \frac{\partial\, \mathrm{LSE}_n}{\partial (\mathbf{C}^\top \mathbf{E})_{k,n}} = \frac{\exp\left((\mathbf{C}^\top \mathbf{E})_{k,n}\right)}{\sum_j \exp\left((\mathbf{C}^\top \mathbf{E})_{j,n}\right)} = \exp\left((\mathbf{C}^\top \mathbf{E})_{k,n} - \mathrm{LSE}_n\right)$

Производная LSE по логиту – это softmax. Назовём её S: её и придётся пересчитывать поблочно.

# Алгоритм 3 – backward

$$\nabla \mathbf{E}^\top = (\mathbf{S} \cdot \nabla \mathrm{LSE})\,\mathbf{C} \qquad \nabla \mathbf{C}^\top = (\mathbf{S} \cdot \nabla \mathrm{LSE})^\top \mathbf{E}$$

**Algorithm 3** Memory-efficient linear-log-sum-exp, backward pass

**Inputs:** $\mathbf{E} \in \mathbb{R}^{D \times N}$, $\mathbf{C} \in \mathbb{R}^{D \times |V|}$, $\mathrm{LSE} \in \mathbb{R}^N$, and $\nabla\mathrm{LSE} \in \mathbb{R}^N$.
Block sizes $N_B$, $V_B$, and $D_B$.
Accuracy threshold $\varepsilon$.

**Outputs:** $\nabla\mathbf{E} \in \mathbb{R}^{D \times N}$, $\nabla\mathbf{C} \in \mathbb{R}^{D \times |V|}$

**for** all pairs of blocks $\mathbf{E}_n$, $\mathbf{C}_v$ **do** ▷ Divide $\mathbf{E}$ and $\mathbf{C}$ into blocks of size $D \times N_B$ and $D \times V_B$
  $\mathbf{A}_{nv} = \mathbf{0}_{V_B \times N_B}$ ▷ Zero matrix of size $V_B \times N_B$ in on-chip SRAM
  **for** blocks $\mathbf{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbf{E}_n$ and $\mathbf{C}_v$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\mathbf{A}_{nv} \mathrel{+}= \mathbf{C}_{v,d}^\top \cdot \mathbf{E}_{n,d}$ ▷ Blockwise matrix multiplication
  **end for**
  $\mathbf{S}_{nv} = \exp(\mathbf{A}_{nv} - \mathrm{LSE}_n)$ ▷ Compute the softmax
  **if** all$(\mathbf{S}_{nv} < \varepsilon)$ **then**
    **skip** ▷ Skip computation if below desired numerical precision
  **end if**
  **for** blocks $\mathbf{E}_{n,d}$, $\mathbf{C}_{v,d}$ **do** ▷ Divide $\mathbf{E}_n$ and $\mathbf{C}_m$ into blocks of $D_B \times N_B$ and $D_B \times V_B$
    $\nabla\mathbf{E}_{n,d}^\top \mathrel{+}= (\mathbf{S}_{nv} \cdot \nabla\mathrm{LSE}_n)\,\mathbf{C}_{v,d}$ ▷ Locking thread-safe gradient update
    $\nabla\mathbf{C}_{v,d}^\top \mathrel{+}= (\mathbf{S}_{nv} \cdot \nabla\mathrm{LSE}_n)^\top \mathbf{E}_{n,d}$ ▷ Locking thread-safe gradient update
  **end for**
**end for**

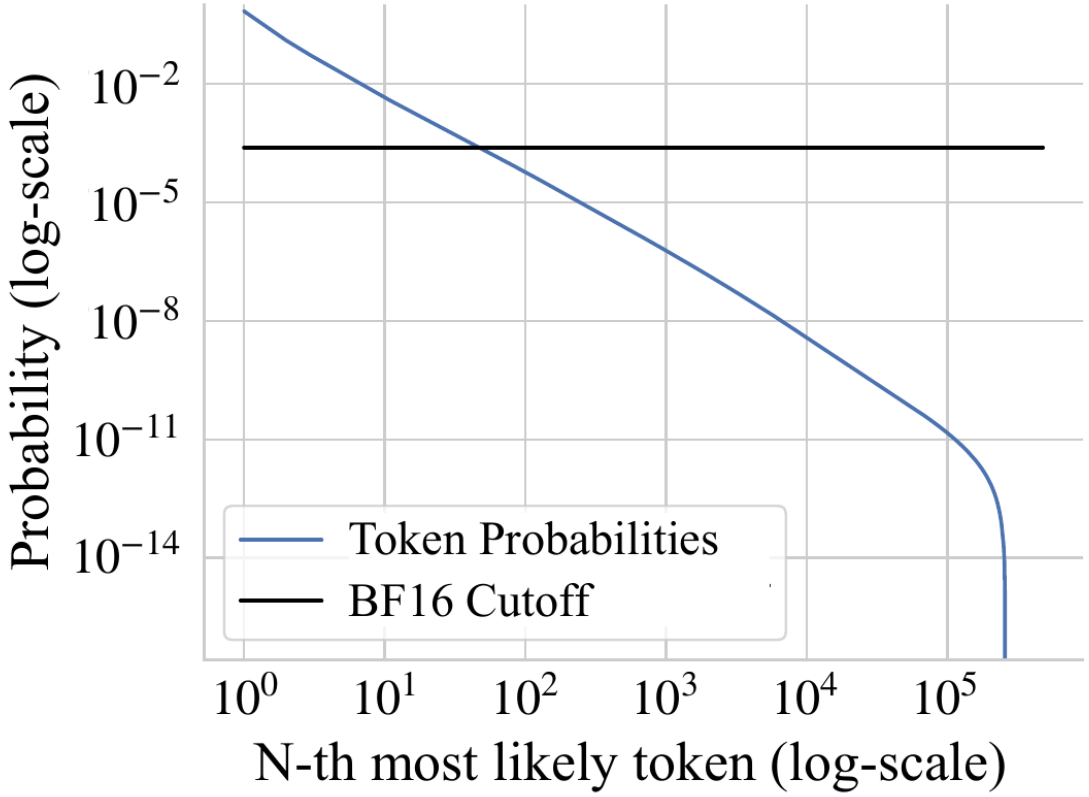
## Gradient filtering

- В bf16 у мантиссы 7 бит: всё, что меньше ε = $2^{-12} \approx 0.000244$, при сложении зануляется
- Softmax очень разреженный: ненулевых меньше 0.02%
- Если весь блок $S_{nv} < \varepsilon$, ядро его пропускает – backward ≈ в 3.5 раза быстрее

## Vocabulary sorting

- На forward токены сортируются по среднему логиту – значимые собираются в одних блоках, пустые с пустыми

# Почему можно пропускать блоки




**Вероятности тают очень быстро**

Средняя вероятность i-го по рангу токена. Уже к ~50-му $p < 2^{-12}$ – ниже точности bf16, такой вклад всё равно потеряется при сложении.

**Абляция, Table 1 (Gemma 2 2B)**

Gradient: 100 мс -> **314 мс** без фильтрации

Gradient: 100 мс -> **115 мс** без сортировки словаря

# Реализации CCE в apple/ml-cross-entropy

**Пропуск блоков** — gradient filtering из Алгоритма 3. Варианты отличаются тем, для какого градиента он включён и в какой точности копятся градиенты.

| реализация | пропуск в ∇E | пропуск в ∇C | накопление | зачем |
|---|---|---|---|---|
| **cce** | да | да | bf16 | **по умолчанию, fine-tuning** |
| `cce_kahan_full_c` | да | **нет** | fp32 / Kahan | pretraining |
| `cce_kahan_full_e` | **нет** | да | fp32 / Kahan | абляция |
| `cce_exact` | **нет** | **нет** | fp32 / Kahan | эталон, проверка |
| `torch_compile` | — | — | — | без Triton (macOS, CPU) |

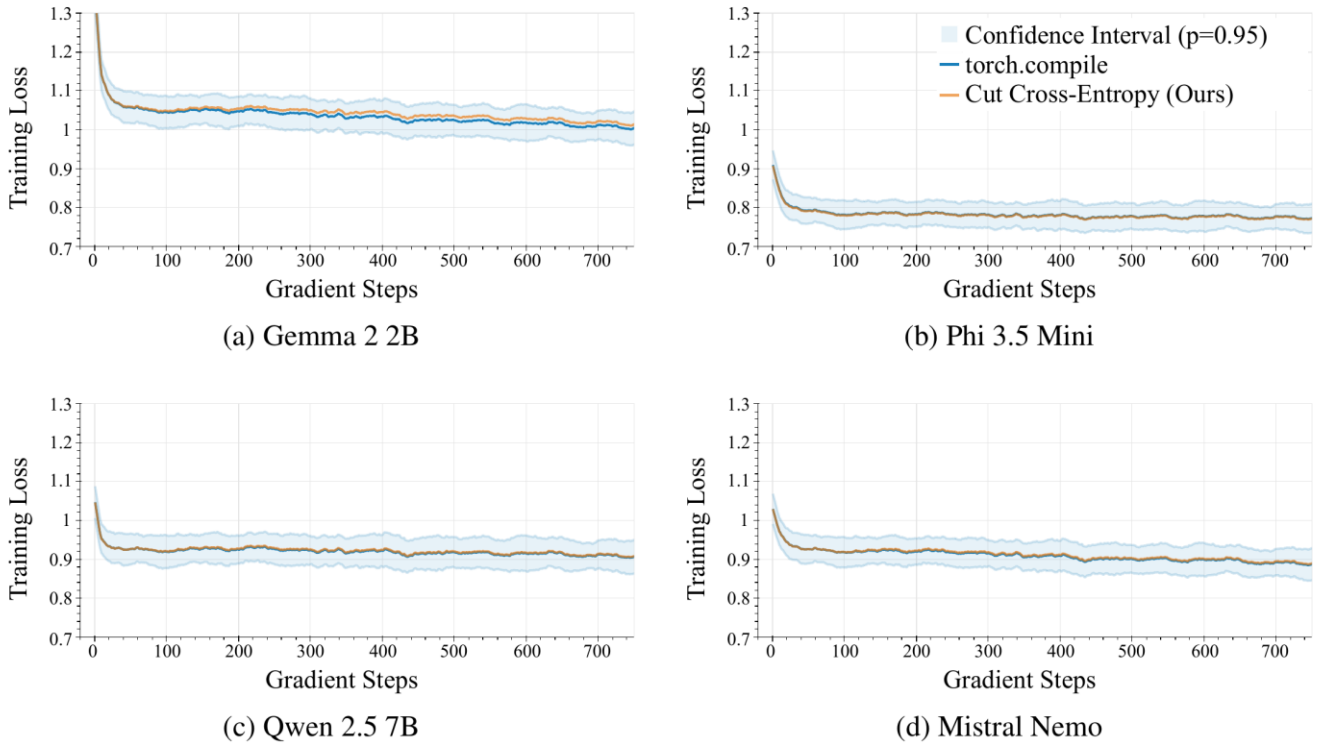
# Другие концепции

| | | |
|---|---|---|
| **Baseline** | логиты целиком + upcast в fp32 (HF, TorchTune по умолчанию) | **O(N · \|V\|)** |
| **torch.compile** | фьюзит softmax и CE, но логиты всё равно в HBM | **O(N · \|V\|)** |
| **TorchTune chunked** | режет токены на 8 чанков + compile | **O(N · \|V\| / k)** |
| **Liger Kernel** | чанки, loss и градиент считаются за один проход | **O(N · D)** |
| **CCE** | блоки в SRAM, recompute в backward, фильтрация | **O(N + \|V\|)** |

# Результаты: Gemma 2 (2B), 8 192 токена

| | | Loss | | Gradient | | Loss+Gradient | |
|---|---|---|---|---|---|---|---|
| | Method | Memory | Time | Memory | Time | Memory | Time |
| | Lower bound | 0.004 MB | | 1,161 MB | | 1,161 MB | |
| 1) | CCE (Ours) | **1 MB** | **46 ms** | **1,163 MB** | 100 ms | **1,164 MB** | 145 ms |
| 2) | Liger Kernels (Hsu et al., 2024)[2] | 1,474 MB | 304 ms | | | 1,474 MB | 304 ms |
| 3) | Torch Tune Team (2024) (8 chunks) | 8,000 MB | 55 ms | 1,630 MB | 115 ms | 9,631 MB | 169 ms |
| 4) | `torch.compile` | 4,000 MB | 49 ms | 12,000 MB | **92 ms** | 16,000 MB | **143 ms** |
| 5) | Baseline | 24,000 MB | 82 ms | 16,000 MB | 122 ms | 28,000 MB | 208 ms |
| 6) | CCE (No Vocab Sorting) | 0.09 MB | 45 ms | 1,162 MB | 115 ms | 1,162 MB | 159 ms |
| 7) | CCE (No Grad. Filter) | 0.09 MB | 45 ms | 1,163 MB | 314 ms | 1,162 MB | 357 ms |
| 8) | CCE-Kahan | 1 MB | 47 ms | 2,325 MB | 114 ms | 2,326 MB | 160 ms |
| 9) | CCE-Kahan-FullC | 1 MB | 47 ms | 2,326 MB | 268 ms | 2,326 MB | 313 ms |
| 10) | CCE-Kahan-FullE | 1 MB | 47 ms | 2,326 MB | 247 ms | 2,326 MB | 292 ms |

# Сохранение стабильности обучения




(a) Gemma 2 2B (b) Phi 3.5 Mini

(c) Qwen 2.5 7B (d) Mistral Nemo

**Fine-tuning**

4 модели на Alpaca, 5 сидов: кривые CCE и torch.compile совпадают.

**Pretraining**

Обычный CCE теряет в perplexity: фильтрация ∇C и накопление в bf16.

**CCE-Kahan-FullC** совпадает с torch.compile.

# Выводы

| | |
|---|---|
| Память | loss + gradient: 28 ГБ → 1.16 ГБ — нижняя граница (только ∇E и ∇C) |
| Скорость | на уровне torch.compile; выигрыш меньше, когда \|V\| / D маленькое |
| Точность | fine-tuning без потерь; для pretraining нужен Kahan-FullC |
| Применение | одна функция вместо головы + CE, уже в Unsloth и Axolotl |